

Proposta de Apresentação (Slide-a-Slide)

SmartCitySystem: Um Gêmeo Digital para Simulação Urbana e Análise de Dados

Aqui está uma estrutura de 16 slides seguindo o modelo da sua professora (a.1 a a.7).

Slide 1: Capa (Início do a.1)

- **Título:** SmartCitySystem: Um Gêmeo Digital para Simulação Urbana e Análise de Dados de Mineração
- **Subtítulo:** Integração de Simulação 3D (Unity), Dados Reais (INMET) e Modelos de Machine Learning (Python/Node.js)
- **Conteúdo:**
 - Seu Nome Completo
 - Nome da Professora
 - Disciplina / Nome do Curso
- **Visual:** Um screenshot de alta qualidade da sua cena 3D da Unity. * **O que Dizer (Roteiro):** "Bom dia a todos. Meu projeto é o SmartCitySystem. O objetivo deste trabalho foi construir um Gêmeo Digital funcional, capaz não apenas de simular uma cidade em 3D, mas também de integrar dados do mundo real e aplicar modelos de mineração de dados para gerar previsões e detectar anomalias."

Slide 2: Introdução e Contexto (a.1)

- **Título:** O que é um Gêmeo Digital?
- **Conteúdo:**
 - **Definição:** Um Gêmeo Digital (Digital Twin) é uma réplica virtual de um sistema físico (um objeto, um processo ou, neste caso, uma cidade).
 - **Por que Cidades?** Cidades são sistemas complexos com milhões de variáveis interconectadas (tráfego, clima, poluição, pessoas).
 - **Motivação:** A capacidade de testar cenários, prever problemas (como congestionamentos ou picos de poluição) e planejar mudanças em um ambiente virtual, sem custos ou riscos do mundo real.
- **Visual:** Um diagrama simples: [Mundo Real] -> (Coleta de Dados) -> [Gêmeo Digital] -> (Análise e Simulação) -> [Insights para o Mundo Real].
- **O que Dizer (Roteiro):** "Para contextualizar, um Gêmeo Digital é um clone virtual de um sistema. A motivação do meu trabalho foi aplicar esse conceito a uma cidade, que é um dos sistemas mais complexos que temos. O valor disso está em poder testar cenários: 'O que acontece se esta rua for fechada?' ou 'Qual será a qualidade do ar amanhã?'."

Slide 3: Problema e Objetivos (a.1)

- **Título:** Problema e Objetivos

- **Conteúdo:**
 - **Problema:** Como integrar, em uma única plataforma, uma simulação 3D interativa (visual) com fontes de dados heterogêneas (climáticos) e modelos preditivos de ML (tráfego e poluição)?
 - **Objetivo Geral:** Desenvolver o "SmartCitySystem", uma plataforma coesa que une Unity 3D, Node.js, Python e MongoDB.
 - **Objetivos Específicos (Nossas Atividades):**
 1. Implementar um replay de dados climáticos históricos (INMET) sob-demanda.
 2. (Atividade 3) Aplicar Mineração de Dados para **Deteção de Outliers** no tráfego (usando LOF).
 3. (Atividade 2 & 4) Aplicar Mineração de Dados para **Classificação** da Qualidade do Ar (usando RandomForest e SMOTE).
 - **Visual:** Ícones para "Tráfego", "Clima", "Poluição" e "Machine Learning".
 - **O que Dizer (Roteiro):** "O problema central é a integração. Meu objetivo foi construir um sistema que resolvesse isso, focando em três recursos principais, que são as minhas atividades da disciplina: um replay de clima, uma detecção de outlier para tráfego e uma classificação de qualidade do ar."

Slide 4: Trabalhos Relacionados (a.2)

- **Título:** Trabalhos Relacionados
- **Conteúdo:**
 - **Simulação 3D e Gêmeos Digitais:** Muitos projetos usam Unity para simulação (ex: Unity, 2025; Espona, 2023), mas historicamente focam na indústria (robótica, manufatura) e não na integração de dados ambientais urbanos em tempo real.
 - **Análise de Dados de Tráfego:** Artigos (ex: Cruz et al., 2017) aplicam algoritmos como o LOF (Local Outlier Factor) para detectar anomalias em dados de tráfego, mas geralmente em análises *offline*, sem uma visualização 3D integrada.
 - **Análise de Qualidade do Ar:** Trabalhos (ex: Vianna Jr. et al., 2025) usam ML para classificar dados da CETESB, e outros (Della-Justina, 2023) focam especificamente no desafio de **dados desbalanceados** (como fizemos com o SMOTE).
 - **A Lacuna (Nosso Projeto):** A literatura carece de projetos que unam essas três frentes: (1) simulação 3D interativa, (2) detecção de outlier em tempo real e (3) classificação de qualidade do ar em um único sistema coeso (Unity + Node.js + Python).
 - **Visual:** Logos do "Google Scholar", "SciELO", "IEEE Xplore".
 - **O que Dizer (Roteiro):** "Na minha revisão, encontrei trabalhos que focavam ou na simulação 3D, ou na análise de tráfego, ou na análise de poluição. A lacuna que meu projeto preenche é a *integração* de todas essas frentes em uma plataforma única e funcional."

Slide 5: Fundamentos (a.3) - Conceitos Chave

- **Título:** Fundamentos: Arquitetura e Conceitos
- **Conteúdo:**
 - **Arquitetura Cliente-Servidor:**

- **Cliente (Frontend):** Unity 3D (para visualização e interação).
- **Servidor (Backend):** Node.js (API "Maestro" que gerencia as requisições).
- **Microserviços de ML via `child_process` :**
 - Em vez de um servidor Flask/Python separado, o Node.js executa scripts Python (`.py`) sob-demanda.
 - Vantagem: Mais leve, mais simples de hospedar (apenas 1 servidor) e eficiente.
- **Banco de Dados:** MongoDB (Banco NoSQL ideal para armazenar os dados JSON dos nossos catálogos e sensores).
- **Visual:** Um diagrama simples da arquitetura (ver Slide 7 para o diagrama completo).
- **O que Dizer (Roteiro):** "Para o projeto funcionar, usei alguns conceitos-chave. A Unity é o cliente. O Node.js é o cérebro (servidor). E o mais importante, para o Machine Learning, usei o Node.js para *chamar* scripts Python diretamente, o que é uma arquitetura muito leve e eficiente."

Slide 6: Fundamentos (a.3) - Mineração de Dados

- **Título:** Fundamentos: Nossas Atividades de Mineração
- **Conteúdo:**
 - **Atividade 3: Detecção de Outliers (LOF):**
 - O que é? Um algoritmo que encontra anomalias.
 - Por que LOF (Local Outlier Factor)? Ele é excelente em encontrar "outliers de densidade".
Ex: Um carro parado (`avgTimeOnSensor` alto) em um tráfego que flui (`vehicleCount` baixo) é uma anomalia de densidade.
 - **Atividade 2: Classificação (Random Forest):**
 - O que é? Um modelo que aprende a prever uma classe (ex: "Boa", "Moderada", "Alerta").
 - **Atividade 4: Dados Desbalanceados (SMOTE):**
 - O que é? O problema de ter poucos exemplos de classes raras (ex: 100 dias "Bons" para cada 1 dia "Alerta").
 - Por que SMOTE? Ele cria "exemplos sintéticos" das classes raras, forçando o modelo a aprender a detectá-las.
- **Visual:**
 - 1. Um gráfico conceitual do LOF (pontos normais vs. anômalos).
 - 2. **(NOVO - Inspirado na Bianca)** Um gráfico de barras "Antes/Depois do SMOTE" (tirado dos seus logs de treino) para mostrar visualmente a Atividade 4. * **O que Dizer (Roteiro):** "O núcleo do trabalho são essas três técnicas de mineração de dados. Para o tráfego, usei o LOF. Para a qualidade do ar, usei o Random Forest para classificar, e o SMOTE para corrigir o desbalanceamento dos dados, como vemos neste gráfico."

Slide 7: Metodologia (a.4) - Arquitetura do Sistema

- **Título:** Metodologia: Arquitetura Completa do Sistema
- **Conteúdo:** (Foco no fluxo de dados)
- **Visual: (ESSENCIAL)** Um diagrama de arquitetura claro (como o do Maicon).

- **[CLIENTE (Unity)]** -> (JSON via HTTP) -> **[BACKEND (Node.js API)]**
- **[BACKEND (Node.js API)]** <-> **[BANCO (MongoDB)]** (Para salvar/ler dados dos sensores e catálogos)
- **[BACKEND (Node.js API)]** -> (Chama `child_process`) -> **[ML (Python Scripts)]**
- **[ML (Python Scripts)]** -> (Lê/Salva `.joblib`) -> **[Modelos Treinados]**
- **O que Dizer (Roteiro):** "Esta é a arquitetura completa do sistema. A Unity (cliente) só conversa com a API Node.js. O Node.js é o 'maestro': ele busca dados no MongoDB e, quando precisa de inteligência, ele executa um script Python, pega a resposta (JSON) e a devolve para a Unity. Tudo de forma assíncrona."

Slide 8: Metodologia (a.4) - Pipeline de Dados

- **Título:** Metodologia: Pipelines de Dados
- **Conteúdo:** (Dividido pelos 3 Recursos que implementamos)
 - **Pipeline 1: Clima (INMET)**
 1. Catalogador (`build_catalog.js`) lê ZIPs e salva no Mongo.
 2. Unity (UI) lê o catálogo via API (`/filters/years ...`).
 - **Pipeline 2: Outlier (Tráfego)**
 1. Unity (`VehicleCounterSensor`) gera dados e salva no Mongo.
 2. API (`/ml/train/outlier`) chama `train_outlier_model.py` .
 - **Pipeline 3: Qualidade do Ar (Qualar)**
 1. Catalogador (`build_qualar_catalog.js`) lê CSVs locais e salva no Mongo.
 2. Unity (UI) lê o catálogo via API (`/filters/qualar/...`).
- **Visual: (NOVO - Inspirado na Bianca)** Um fluxograma visual de um dos pipelines (ex: o Pipeline 3).
 - [Pasta `qualar_data`] -> `build_qualar_catalog.js` -> [Mongo `qualar_file_catalog`] -> API (`/filters/...`) -> [UI da Unity (Dropdowns)]
- **O que Dizer (Roteiro):** "A metodologia foi dividida em três pipelines de dados. Este fluxograma, por exemplo, mostra o pipeline da Qualidade do Ar: o catalogador lê os arquivos CSV, salva no Mongo, a API lê o Mongo, e a Unity consome a API para popular os filtros."

Slide 9: Experimentos e Resultados (a.5) - Recurso 1: Replay de Clima

- **Título:** Resultados: Recurso 1 - Replay de Clima (INMET)
- **Conteúdo:**
 - **Desafio:** Como lidar com *gigabytes* de arquivos CSV do INMET sem travar o servidor ou a Unity?
 - **Solução (Catálogo Sob-Demanda):** Implementamos um catalogador que pré-processa apenas os *nomes* dos arquivos. A UI filtra instantaneamente.
 - **Resultado:** A API importa *apenas* o arquivo CSV necessário, sob-demanda, quando o usuário clica. O `SimulationClock` então lê esses dados importados e ajusta o ambiente (chuva, céu).

- **Visual:** Um screenshot da UI de filtros do INMET na Unity. Opcional: um screenshot da cena com chuva (se você tiver).
- **O que Dizer (Roteiro):** "O primeiro resultado foi o Replay de Clima. O desafio eram os dados massivos. A solução foi um catálogo sob-demanda. Como veremos na demo, a UI filtra anos e regiões instantaneamente, e o sistema só baixa o dado quando o usuário pede."

Slide 10: Experimentos e Resultados (a.5) - Recurso 2: Detecção de Outlier

- **Título:** Resultados: Recurso 2 - Detecção de Outlier de Tráfego (Ativ. 3)
- **Conteúdo:**
 - **Objetivo:** Detectar anomalias de tráfego (congestionamentos) em tempo real.
 - **Experimento:**
 1. Sensores 3D na Unity (`VehicleCounterSensor`) coletaram dados de tráfego "normal" dos carros-robô.
 2. Treinamos o modelo LOF (Local Outlier Factor) com esses dados.
 3. **Teste de Anomalia:** Forçamos um carro-robô a parar dentro do sensor (desativando seu script de IA).
 - **Resultado:** O sistema detectou com sucesso o "outlier" (carro parado) e instanciou um "prefab" de incidente (carro quebrado) no local.
- **Visual:** Um screenshot do `incidentPrefab` (o carro quebrado/cones) na cena da Unity.
- **O que Dizer (Roteiro):** "Para a Atividade 3, o objetivo era detectar outliers no tráfego. Eu treinei o modelo com dados normais e, para testar, forcei uma anomalia. O resultado foi que o sistema identificou o carro parado e criou um incidente visual na simulação."

Slide 11: Experimentos e Resultados (a.5) - Recurso 3: Classificação

- **Título:** Resultados: Recurso 3 - Classificação de Qualidade do Ar (Ativ. 2 & 4)
- **Conteúdo:**
 - **Objetivo:** Prever a qualidade do ar (Boa, Moderada, Alerta) baseada no contexto (hora, dia, mês).
 - **Desafio:** A API oficial da CETESB se mostrou inacessível (bloqueando todas as requisições de filtro).
 - **Solução (Plano B):** Implementamos um pipeline de dados local. Criamos um catálogo (`build_qualar_catalog.js`) para ler arquivos CSV locais (ex: `qualar_2024_bauru_mp10.csv`).
 - **Resultado:** A UI filtra os dados locais, importa-os, e o `SimulationClock` chama a API de predição, exibindo a classe prevista ("Boa", "Moderada") e a confiança na UI.
- **Visual:** Screenshot da UI de filtros do Qualar (com Ano, Estação, Parâmetro). Screenshot do texto de predição ("Qualidade do Ar: Boa (92%)") na UI da Unity.
- **O que Dizer (Roteiro):** "Para a Atividade de Classificação, tivemos um desafio: a API da CETESB estava bloqueada. A solução foi criar um pipeline de dados 100% local, baseado em arquivos CSV. O resultado é que o sistema importa esses dados e o relógio da simulação agora prevê a qualidade do ar a cada hora."

Slide 12: Discussão (a.5) - O Desafio dos Dados Desbalanceados

- **Título:** Discussão: O Impacto do SMOTE (Atividade 4)
- **Conteúdo:**
 - **Problema (Dados Reais):** Nossos dados de qualidade do ar eram altamente desbalanceados. (Ex: Boa: 5208 registros, vs Alerta: 149 registros).
 - **Solução (SMOTE):** Aplicamos a técnica SMOTE, que criou amostras sintéticas das classes "Moderada" e "Alerta" até que todas tivessem 5208 registros.
 - **Discussão (Resultados):**
 - O modelo treinado (com SMOTE) teve uma acurácia geral de 83%.
 - **Insight:** O F1-Score (que mede a precisão) foi excelente para "Boa" (0.91), mas muito baixo para "Alerta" (0.14).
 - **Conclusão Acadêmica:** Isso mostra que, mesmo com SMOTE, prever eventos raros é um desafio complexo. O modelo aprendeu a "apostar no seguro" (prever "Boa"), indicando que features adicionais (como dados de clima) seriam necessárias para melhorar a predição de alertas.
- **Visual: (ESSENCIAL - Inspirado em Joao e Bianca)**
 - 1. O gráfico de barras "Antes/Depois do SMOTE" (do log de treino).
 - 2. A tabela do `classification_report` (do log de treino) para mostrar os números (0.91 vs 0.14).
- **O que Dizer (Roteiro):** "Aqui está o resultado da Atividade 4. À esquerda, os dados originais. À direita, os dados balanceados pelo SMOTE. Mas o mais interessante está nesta tabela de resultados: o modelo ficou ótimo em prever 'Boa', mas péssimo em prever 'Alerta'. Isso não é um erro, é uma descoberta: o modelo aprendeu que 'chutar' Boa é a aposta mais segura. Isso mostra que só o SMOTE não é suficiente."

Slide 13: Demonstração ao Vivo (a.5)

- **Título:** Demonstração ao Vivo
- **Conteúdo:** (Este slide é apenas um placeholder para você iniciar sua demo)
- **Visual:** Um screenshot grande e limpo da sua cena principal da Unity.
- **O que Dizer (Roteiro):** "Agora, gostaria de fazer uma demonstração ao vivo do sistema que acabamos de discutir." (Siga o roteiro do arquivo `roteiro_demo_aovivo.md`)

Slide 14: Conclusão (a.6)

- **Título:** Conclusão e Trabalhos Futuros
- **Conteúdo:**
 - **Trabalho Realizado:** Foi construída com sucesso uma plataforma de Gêmeo Digital funcional, integrando Unity, Node.js, Python e MongoDB.
 - **Resultados (Atividades):**
 1. Replay de clima (INMET) sob-demanda foi implementado.
 2. Detecção de Outlier (LOF) em tempo real foi validada.

3. Classificação (RandomForest) e Balanceamento (SMOTE) foram aplicados com sucesso.

- **Trabalhos Futuros:**

1. Usar a predição (de clima e poluição) para *alterar* a simulação (ex: adicionar névoa de poluição, mudar a rota dos carros).
2. **(NOVO - Sua Ideia)** Criar um painel de visualização (dashboard) em HTML que gere gráficos (`matplotlib`) automaticamente a partir dos resultados.
3. **(NOVO - Sua Ideia)** Integrar o Gemini para gerar explicações automáticas em linguagem natural desses gráficos.

- **Visual:** (Apenas texto, bem formatado).

- **O que Dizer (Roteiro):** "Em conclusão, o projeto atingiu todos os objetivos, criando um sistema integrado. As atividades de mineração de dados foram validadas. Como trabalhos futuros, eu gostaria de usar as predições para mudar a simulação e, como sugerido, gerar um dashboard em HTML com explicações de IA."

Slide 15: Bibliografia (a.7)

- **Título:** Bibliografia
- **Conteúdo:** (Liste os artigos da "Trabalhos Relacionados" e bibliotecas-chave)
 - CRUZ, A. B., et al. (2017). "Detecção de Anomalias no Transporte Rodoviário Urbano". *Anais do Simpósio Brasileiro de Banco de Dados (SBBD)*.
 - DELLA-JUSTINA, H. M. (2023). "Avaliação de Técnicas de Classificação Para Dados Desbalanceados". *Acervo Digital UFPR*.
 - VIANNA JR, A. S., et al. (2025). "Analisando Dados de Qualidade do Ar por Machine Learning". *Periodicos FURG*.
 - Unity Technologies. (2025). "Construa Gêmeos Digitais Industriais".
 - Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python".
 - Lemaître, G. et al. (2017). "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets".
- **Visual:** (Nenhum).

Slide 16: Fim

- **Título:** Obrigado
- **Conteúdo:**
 - Perguntas?
 - (Seu e-mail de contato)
- **Visual:** (O mesmo da capa ou um screenshot da cena).